REVISION CHEAT SHEET: UNIT TESTING / TESTABLE C#

# Unit Testing and Testable C#

Interview-ready notes · examples · C# · diagrams · clickable index

## INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.


1. Why Testing Matters
2. What Is a Unit Test?
3. Unit Test vs Integration Test
Unit Test
Integration Test
4. Unit vs Integration vs End-to-End
5. The Testing Pyramid
6. What Makes a Good Unit Test?
Fast
Isolated
Repeatable
Self-Checking
Timely
7. Arrange, Act, Assert — AAA
8. Arrange
9. Act
10. Assert
11. One Behavior Per Test
12. Does One Test Mean One Assert?
13. Test Naming
14. xUnit in Modern .NET
15. Basic xUnit Test
16. Fact vs Theory
17. Theory with InlineData
18. Theory with MemberData
19. Avoid Loops as Test-Case Engines
20. Common Assertions
21. Test Exceptions
22. Async Tests
23. Testing Async Exceptions
24. Never Block on Async Code in Tests
25. Testing Cancellation
PART 2 — TEST DOUBLES
26. What Is a Test Double?
27. Dummy
28. Stub
29. Fake
30. Spy
31. Mock
32. Stub vs Mock
33. Fake vs Stub
34. State-Based Testing
35. Interaction-Based Testing
36. Do Not Over-Mock
37. Test Behavior, Not Implementation
PART 3 — TESTABLE C# DESIGN
38. Explicit Dependencies
39. Static Time Is Hard to Test
40. Randomness Should Be Controllable
41. File System and Network Boundaries
42. Pure Functions Are Easy to Test
43. Encapsulation Helps Testing
44. Avoid Hidden Global State
PART 4 — DOMAIN TESTING
45. Domain Tests Should Be the Easiest Tests
46. Test Boundary Values
47. Positive and Negative Paths
PART 5 — APPLICATION SERVICE TESTING
48. Application Service Example
49. Test Successful Application Flow
50. Test Not-Found Flow
51. What Not to Test in Application Unit Tests
PART 6 — ASP.NET CORE TESTING
52. Controller Unit Tests
53. Controller Unit Test
54. ASP.NET Core Integration Testing

55. What WebApplicationFactory Does Not Mean

PART 7 — EF CORE TESTING

56. EF Core Testing Requires Special Care

57. Best Confidence: Test the Real Database Provider

58. Why SQLite Is Not the Same as SQL Server

59. EF Core InMemory Provider

60. Do Not Mock DbSet Queries

61. Repository as Test Seam

62. Repository Tests Still Need Real Database Tests

63. Database Test Isolation

64. Do Not Depend on Test Order

PART 8 — TEST FIXTURES AND LIFECYCLE

65. xUnit Creates Fresh Test-Class Instances

66. Class Fixture

67. Do Not Share Mutable Test State Carelessly

PART 9 — PARALLEL TESTING

68. Tests May Run in Parallel

69. Parallel-Safe Tests

PART 10 — CODE COVERAGE

70. What Is Code Coverage?

71. Running Coverage

72. 100% Coverage Does Not Mean Correct Software

73. Branch Coverage

74. What Should You Aim to Cover?

PART 11 — TDD

75. Test-Driven Development

- Red
- Green
- Refactor

76. TDD Is Not the Same as Unit Testing

PART 12 — BRITTLE AND FLAKY TESTS

77. What Is a Brittle Test?

78. What Is a Flaky Test?

79. Never Fix Flakiness with Random Sleeps

80. Test Timeouts

PART 13 — SECURITY TESTING

81. Test Authorization Boundaries

82. Repository Tenant Test

83. Do Not Put Secrets in Test Data

PART 14 — PERFORMANCE AND TEST DESIGN

84. Unit Tests Should Be Fast Enough to Run Constantly

85. Integration Tests Can Still Be Fast

PART 15 — TEST B UNIT TESTING / TESTABLE C#

86. The Object Mother Problem

87. GiftTestBuilder

88. Builder Must Not Hide Important Setup

PART 16 — WHAT NOT TO TEST

89. Do Not Test Framework Code

90. Do Not Test Private Methods Directly

91. Do Not Test Trivial Auto-Properties

PART 17 — FULL FUNDRAISING TESTING STRATEGY

92. Domain Layer

93. Application Layer

94. Infrastructure Layer

95. API Layer

Diagram 1 — Testing Boundaries

PART 18 — COMMON MISTAKES

- Mistake 1 — Unit Test Uses Real Database
- Mistake 2 — Mock Everything
- Mistake 3 — Testing Implementation Calls
- Mistake 4 — One Giant Test
- Mistake 5 — Shared Mutable Static Data
- Mistake 6 — async void Tests
- Mistake 7 — .Result and .Wait()
- Mistake 8 — Testing EF Queries with InMemory Provider as Proof of SQL Correctness
- Mistake 9 — SQLite Test Assumed Equivalent to SQL Server
- Mistake 10 — Chasing 100% Coverage
- Mistake 11 — Random Test Data Everywhere
- Mistake 12 — Time-Based Sleeps

PART 19 — BEST PRACTICES

PART 20 — PRACTICAL EXERCISES

- Exercise 1 — Basic Domain Test
- Exercise 2 — Allocation Boundary
- Exercise 3 — Complete Gift
- Exercise 4 — Stub Repository
- Exercise 5 — Spy Outbox
- Exercise 6 — Fake Clock
- Exercise 7 — Async Exception
- Exercise 8 — SQL Server Repository Integration Test
- Exercise 9 — API Test
- Exercise 10 — Theory Conversion
- Exercise 11 — Coverage Review
- Exercise 12 — Flaky Test Review

PART 21 — EXACTLY 25 INTERVIEW QUESTIONS

1. What is a unit test?
2. What is the difference between unit testing and integration testing?
3. What is the AAA pattern?
4. What makes a good unit-test name?
5. What is the difference between [Fact] and [Theory] in xUnit?
6. What is a test double?
7. What is the difference between a stub and a mock?
8. What is the difference between a fake and a mock?
9. Why is dependency injection useful for testability?
10. Why should DateTime.UtcNow often be abstracted in important business logic?
11. How do you test asynchronous methods?
12. How do you test an asynchronous exception with xUnit?
13. What is a brittle test?
14. What is a flaky test?
15. Should every class be mocked in a unit test?
16. Should private r
UNIT TESTING / TESTABLE C#
17. Why is the EF Core InMemory provider not a reliable replacement for SQL Server tests?
18. Is SQLite in-memory equivalent to SQL Server for EF Core testing?
19. Should DbSet queries be mocked?
20. How should Repository code be tested?
21. How should ASP.NET Core controllers be tested?
22. What is WebApplicationFactory<TEntryPoint> ?
23. Does 100% code coverage mean the application is fully tested?
24. What is TDD?
25. How would you test the Fundraising Platform?

**PART 22 — QUICK REVISION**

**PART 23 — COMPLETE TESTING FLOW FOR THE FUNDRAISING PLATFORM**

**FINAL MENTAL MODEL**

## 1. Why Testing Matters

Testing is not simply about proving that code works today.

A good automated test suite helps you answer:

**EXAMPLE / FLOW**

```
Can I safely change this code?

Did this refactoring preserve behavior?

Did I accidentally break an existing business rule?

Can another developer understand what this class promises?

Does my EF Core query actually work against SQL Server?

Does my ASP.NET Core endpoint behave correctly through the real pipeline?
```

Unit testing is especially valuable when combined with good design.

Code that is difficult to test often reveals architectural problems such as:

**EXAMPLE / FLOW**

```
Hidden dependencies
Static global state
Too many responsibilities
Direct infrastructure coupling
Non-deterministic time
Hard-coded network calls
Large God classes
```

Microsoft's current .NET testing guidance describes good unit tests as **fast, isolated, repeatable, self-checking, and timely**, and recommends keeping infrastructure dependencies out of unit tests.

## 2. What Is a Unit Test?

A unit test verifies a **small unit of behavior in isolation**.

For example, your Fundraising domain might contain:

```C#
public sealed class Gift
{
    public decimal TotalAmount { get; }

    public GiftStatus Status { get; private set; }

    public void Complete()
    {
        if (TotalAmount <= 0)
        {
            throw new InvalidOperationException(
                "Gift amount must be positive.");
        }

        Status = GiftStatus.Completed;
    }
}
```

A unit test can verify:

```EXAMPLE / FLOW
Given a valid pending Gift

When Complete() is called

Then Status becomes Completed
```

without requiring:

```EXAMPLE / FLOW
SQL Server
ASP.NET Core server
HTTP request
Message broker
Email provider
```

That isolation is one of the defining characteristics of unit testing.

## 3. Unit Test vs Integration Test

These are different kinds of confidence.

### Unit Test

Tests one small component, usually with dependencies replaced by controlled test doubles.

Example:

```EXAMPLE / FLOW
CompleteGiftService
    ↓
Fake repository
Fake unit of work
Fake clock
```

You are testing:

```
EXAMPLE / FLOW
application-service behavior
```

not:

```
EXAMPLE / FLOW
EF Core
SQL Server
HTTP
```

### Integration Test

An integration test verifies that multiple real components work together.

For example:

```
EXAMPLE / FLOW
GiftRepository
    ↓
EF Core
    ↓
SQL Server
```

or:

```
EXAMPLE / FLOW
HTTP request
    ↓
ASP.NET Core routing
    ↓
Middleware
    ↓
Controller
    ↓
Application
```

Microsoft defines integration tests as exercising two or more components together, often including infrastructure.

## 4. Unit vs Integration vs End-to-End

A useful testing spectrum is:

| Test type | Scope | Speed | Typical dependencies |
|---|---|---|---|
| Unit | Small behavior | Very fast | Test doubles |
| Integration | Several real components | Medium | DB/framework/files/network substitute or real service |
| API integration | Full ASP.NET Core request pipeline | Medium | Test server + application |
| End-to-end | Complete deployed workflow | Slowest | Real system boundaries |
| Load/ performance | System under pressure | Variable | Production-like environment |

No single type replaces the others.

A unit test can prove:



EXAMPLE / FLOW

```
Gift.Complete() enforces its rule.
```

It cannot prove:

EXAMPLE / FLOW

```
EF mapping for Gift.Status works in SQL Server.
```

An EF integration test can prove that.

## 5. The Testing Pyramid

A classic mental model is:

EXAMPLE / FLOW

```
          /\
         /  \
        / E2E\
       /------\
      /Integration\
     /------------\
    /  Unit Tests  \
   /________________\
```

The general idea:

EXAMPLE / FLOW

```
Many fast unit tests

Fewer integration tests

A smaller number of expensive
end-to-end tests
```

This is a guideline rather than a fixed mathematical ratio.

A database-heavy application may reasonably have significant integration coverage because persistence behavior matters greatly.

## 6. What Makes a Good Unit Test?

Microsoft highlights five characteristics:

EXAMPLE / FLOW

```
Fast

Isolated

Repeatable

Self-checking

Timely
```

Let's understand each.

## Fast

A unit test should usually finish in milliseconds.

Bad unit test:

```C#
[Fact]
public async Task CompleteGift_Works()
{
    await Task.Delay(
        TimeSpan.FromSeconds(5));

    // Test.
}
```

A thousand tests like this become painful to run.

Developers then stop running the suite.

## Isolated

A unit test should not depend on:

```EXAMPLE / FLOW
Internet connection
SQL Server
Real clock time
Shared file
External API
Another test
```

unless the test is intentionally an integration test.

## Repeatable

This should not happen:

```EXAMPLE / FLOW
Run 1 → Pass
Run 2 → Fail
Run 3 → Pass
```

with unchanged code.

That is a flaky test.

## Self-Checking

The test itself determines success.

Bad:

```C#
Console.WriteLine(
    result);
```

and then a person visually inspects it.

Good:

```C#
Assert.Equal(
    GiftStatus.Completed,
    gift.Status);
```

**Timely**

Tests should be practical to write and maintain.

If testing a tiny class requires:

EXAMPLE / FLOW

```
15 mocks
complex DI setup
real configuration
reflection
```

the production design may be too coupled.

## 7. Arrange, Act, Assert — AAA

A common unit-test structure is:

EXAMPLE / FLOW

```
Arrange
    ↓
Act
    ↓
Assert
```

Example:

C#

```
[Fact]
public void Complete_WhenGiftIsValid_SetsStatusToCompleted()
{
    // Arrange
    Gift gift =
        GiftTestBuilder
            .CreateValidGift();

    // Act
    gift.Complete();

    // Assert
    Assert.Equal(
        GiftStatus.Completed,
        gift.Status);
}
```

## 8. Arrange

Arrange creates:

EXAMPLE / FLOW

```
Input

System under test

Required test doubles

Expected values
```

Example:

```C#
Gift gift =
    GiftTestBuilder
        .CreateValidGift();
```

Keep Arrange as small as possible.

Microsoft recommends using the minimum input needed to prove the behavior because unnecessary setup makes tests brittle and obscures intent.

## 9. Act

Act performs the behavior being tested.

Example:

```C#
gift.Complete();
```

A good unit test normally has one meaningful action.

This does **not** mean only one method may ever appear after Arrange, but the test should be centered around one behavior.

## 10. Assert

Assert verifies the observable outcome.

```C#
Assert.Equal(
    GiftStatus.Completed,
    gift.Status);
```

Possible observable outcomes include:

```EXAMPLE / FLOW
Return value

State change

Exception

Dependency interaction

Generated event

Collection contents
```

## 11. One Behavior Per Test

Avoid tests such as:

```C#
[Fact]
public void Gift_AllOperations_Work()
{
    gift.AddAllocation(...);

    Assert.Equal(...);
```

```
    gift.Complete();

    Assert.Equal(...);

    gift.Cancel();

    Assert.Equal(...);
}
```

If `Complete()` breaks, the test name does not clearly indicate the failed behavior.

Prefer focused tests:

EXAMPLE / FLOW

```
AddAllocation_WhenValid_AddsAllocation

Complete_WhenFullyAllocated_SetsStatusCompleted

Cancel_WhenPending_SetsStatusCancelled
```

## 12. Does One Test Mean One Assert?

No.

A useful rule is:

> **One behavior per test, not necessarily one assertion per test.**

Suppose creating a Gift guarantees:

EXAMPLE / FLOW

```
Correct amount
Pending status
Correct tenant
```

These may reasonably be asserted together if they represent one construction contract.

Example:

C#

```
Assert.Equal(
    1,
    gift.TenantId);

Assert.Equal(
    100m,
    gift.TotalAmount);

Assert.Equal(
    GiftStatus.Pending,
    gift.Status);
```

Artificially splitting every property into a separate test can make suites noisy.

## 13. Test Naming

Microsoft recommends names that communicate:

EXAMPLE / FLOW

```
Method being tested

Scenario

Expected result
```

A common convention is:

EXAMPLE / FLOW

```
Method_Scenario_ExpectedResult
```

Examples:

EXAMPLE / FLOW

```
Complete_WhenGiftIsFullyAllocated_SetsStatusToCompleted

Complete_WhenAllocationIsIncomplete_ThrowsInvalidOperationException

AddAllocation_WhenAmountExceedsGift_Throws

FindAsync_WhenGiftDoesNotExist_ReturnsNull
```

Another readable style is:

EXAMPLE / FLOW

```
Given_When_Then
```

For example:

EXAMPLE / FLOW

```
GivenIncompleteAllocation_WhenGiftCompletes_ThenThrows
```

Consistency matters more than choosing one universal convention.

## 14. xUnit in Modern .NET

For the examples in these notes, we'll use **xUnit**.

Current xUnit v3 documentation supports modern .NET and continues to use the core concepts:

EXAMPLE / FLOW

```
[Fact]

[Theory]

[InlineData]

[MemberData]

Assert.*
```

The current xUnit v3 getting-started documentation demonstrates running modern xUnit projects with the .NET SDK and describes Facts as invariant tests and Theories as data-driven tests.

## 15. Basic xUnit Test

```C#
public sealed class GiftTests
{
    [Fact]
    public void Complete_WhenValid_SetsCompletedStatus()
    {
        Gift gift =
            GiftTestBuilder
                .CreateValidGift();

        gift.Complete();

        Assert.Equal(
            GiftStatus.Completed,
            gift.Status);
    }
}
```

`[Fact]` identifies a normal test case with no input-data rows.

## 16. Fact vs Theory

Use:

```C#
[Fact]
```

when testing one fixed case.

Use:

```C#
[Theory]
```

when the same behavior should be tested against several inputs.

xUnit describes a Theory as a data-driven test whose method runs once for each supplied data row.

## 17. Theory with InlineData

Suppose Gift amounts must be positive.

```C#
[Theory]
[InlineData(0)]
[InlineData(-1)]
[InlineData(-100)]
public void Create_WhenAmountIsNotPositive_Throws(
    decimal amount)
{
    Assert.Throws<
        ArgumentOutOfRangeException>(
            () =>
                Gift.Create(
                    tenantId:
                        1,
                    constituentId:
                        10,
                    totalAmount:
```

```
                        amount));
}
```



Instead of:

```
EXAMPLE / FLOW
three almost identical tests
```

one Theory expresses the rule clearly.

## 18. Theory with MemberData

`InlineData` is useful for simple constants.

For more complex objects, use:

```
C#
[MemberData]
```

Example:

```
C#
public static TheoryData<
    decimal,
    decimal,
    bool>
    AllocationCases
    =>
    new()
    {
        { 100m, 100m, true },
        { 100m, 50m, true },
        { 100m, 101m, false }
    };

[Theory]
[MemberData(
    nameof(
        AllocationCases))]
public void CanAllocate_ReturnsExpectedResult(
    decimal giftAmount,
    decimal allocationAmount,
    bool expected)
{
    Gift gift =
        Gift.Create(
            1,
            10,
            giftAmount);

    bool actual =
        gift.CanAllocate(
            allocationAmount);

    Assert.Equal(
        expected,
        actual);
}
```

## 19. Avoid Loops as Test-Case Engines

Bad:

```C#
[Fact]
public void AmountTests()
{
    foreach (
        decimal amount
        in new[]
        {
            0m,
            -1m,
            -100m
        })
    {
        Assert.Throws<
            ArgumentOutOfRangeException>(
                () =>
                    Gift.Create(
                        1,
                        10,
                        amount));
    }
}
```

If one case fails, diagnosis is less clear.

Microsoft's unit-testing guidance recommends avoiding control-flow logic such as loops and conditions in tests where separate data-driven cases can express the behavior more directly.

Use:

```C#
[Theory]
```

instead.

## 20. Common Assertions

Typical xUnit assertions include:

```C#
Assert.Equal(
    expected,
    actual);

Assert.NotEqual(
    expected,
    actual);

Assert.True(
    condition);

Assert.False(
    condition);

Assert.Null(
    value);

Assert.NotNull(
```

```
    value);

Assert.Empty(
    collection);

Assert.Single(
    collection);

Assert.Contains(
    expected,
    collection);
```

xUnit's `Assert` class provides assertion APIs for values, collections, exceptions, asynchronous operations, and other test conditions.

## 21. Test Exceptions

Suppose:

C#

```
public void Complete()
{
    if (!IsFullyAllocated)
    {
        throw new
            InvalidOperationException(
                "Gift must be fully allocated.");
    }

    Status = GiftStatus.Completed;
}
```

Test:

C#

```
[Fact]
public void Complete_WhenNotFullyAllocated_Throws()
{
    Gift gift =
        GiftTestBuilder
            .CreatePartiallyAllocatedGift();

    InvalidOperationException exception =
        Assert.Throws<
            InvalidOperationException>(
                gift.Complete);

    Assert.Equal(
        "Gift must be fully allocated.",
        exception.Message);
}
```

Be careful about asserting full exception messages if wording is not part of the contract.

If only exception type matters, asserting the type may create a more resilient test.

## 22. Async Tests

An asynchronous test should normally return:

C#
```
Task
```

Example:

C#
```
[Fact]
public async Task CreateAsync_WhenValid_AddsGift()
{
    FakeGiftRepository repository = new();

    CreateGiftService service =
        new(
            repository);

    await service.CreateAsync(
        CreateValidCommand(),
        CancellationToken.None);

    Assert.Single(
        repository.Gifts);
}
```

Do not write:

C#
```
[Fact]
public async void CreateAsync_WhenValid_AddsGift()
```

Use `Task` so the test framework can properly observe completion and exceptions.

## 23. Testing Async Exceptions

Example production code:

C#
```
public async Task<Gift> GetRequiredAsync(
    long giftId,
    CancellationToken cancellationToken)
{
    Gift? gift =
        await _repository.FindAsync(
            giftId,
            cancellationToken);

    return gift
        ??
        throw new
            GiftNotFoundException(
                giftId);
}
```

Test:

C#
```
[Fact]
public async Task GetRequiredAsync_WhenMissing_Throws()
{
    StubGiftRepository repository =
        new(
            gift:
```

```
                null);

    GiftService service =
        new(
            repository);

    GiftNotFoundException exception =
        await Assert.ThrowsAsync<
            GiftNotFoundException>(
                () =>
                    service.GetRequiredAsync(
                        100,
                        CancellationToken.None));

    Assert.Equal(
        100,
        exception.GiftId);
}
```

Current xUnit v3 `ThrowsAsync<T>` verifies that the awaited delegate throws the expected exception type and returns the captured exception for further assertions.

## 24. Never Block on Async Code in Tests

Avoid:

```C#
service.CreateAsync(...)
    .Result;
```

or:

```C#
service.CreateAsync(...)
    .Wait();
```

Prefer:

```C#
await service.CreateAsync(...);
```

This matches production async behavior and avoids introducing sync-over-async issues into tests.

## 25. Testing Cancellation

Suppose:

```C#
public Task ProcessAsync(
    CancellationToken cancellationToken)
{
    cancellationToken
        .ThrowIfCancellationRequested();

    return Task.CompletedTask;
}
```

Test:

```C#
[Fact]
public async Task ProcessAsync_WhenCancelled_ThrowsCancellation()
{
    using CancellationTokenSource source = new();

    source.Cancel();

    await Assert.ThrowsAnyAsync<
        OperationCanceledException>(
            () =>
                _service.ProcessAsync(
                    source.Token));
}
```

Test cancellation behavior when cancellation is part of the public contract.

Do not add cancellation tests to every tiny method simply because a token exists.

# PART 2 — TEST DOUBLES

## 26. What Is a Test Double?

A test double replaces a dependency during a test.

The general term includes concepts such as:

```EXAMPLE / FLOW
Dummy

Stub

Fake

Spy

Mock
```

These terms sometimes vary slightly between teams/frameworks, but the underlying ideas are useful.

## 27. Dummy

A Dummy exists only because a parameter is required.

Its behavior is not used.

Example:

```C#
new GiftService(
    repository:
        repository,
    logger:
        NullLogger<GiftService>.Instance);
```

If logging is irrelevant to the test, the logger is effectively a dummy dependency.

## 28. Stub

A Stub returns controlled results.

Example:



C#

```
public sealed class StubGiftRepository :
    IGiftRepository
{
    private readonly Gift? _gift;

    public StubGiftRepository(
        Gift? gift)
    {
        _gift = gift;
    }

    public Task<Gift?> FindAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken)
    {
        return Task.FromResult(
            _gift);
    }
}
```

Purpose:

EXAMPLE / FLOW

```
When repository is asked for Gift
    ↓
return this predefined result
```

The test usually does not care whether the stub method was called exactly three times.

## 29. Fake

A Fake is a lightweight working implementation.

Example:

C#

```
public sealed class FakeGiftRepository :
    IGiftRepository
{
    private readonly
        List<Gift>
        _gifts = [];

    public IReadOnlyList<Gift> Gifts =>
        _gifts;

    public Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        _gifts.Add(
            gift);

        return Task.CompletedTask;
    }

    public Task<Gift?> FindAsync(
        long tenantId,
        long giftId,
```

```
        CancellationToken cancellationToken)
    {
        Gift? gift =
            _gifts.SingleOrDefault(
                item =>
                    item.TenantId
                    ==
                    tenantId
                    &&
                    item.GiftId
                    == giftId);

        return Task.FromResult(
            gift);
    }
}
```

This behaves like a tiny in-memory repository but is not production persistence.

## 30. Spy

A Spy records what happened so the test can inspect it.

C#

```
public sealed class SpyReceiptSender :
    IReceiptSender
{
    public int SendCount
    {
        get;
        private set;
    }

    public Gift? LastGift
    {
        get;
        private set;
    }

    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        SendCount++;

        LastGift = gift;

        return Task.CompletedTask;
    }
}
```

Test:

C#

```
Assert.Equal(
    1,
    receiptSender.SendCount);

Assert.Same(
    gift,
    receiptSender.LastGift);
```

## 31. Mock

A Mock is generally configured with expected behavior/interactions and can verify that those interactions occurred.

Conceptually:

**EXAMPLE / FLOW**

```
Expect repository.AddAsync(gift)
    ↓
Execute service
    ↓
Verify expected interaction occurred
```

Mocking libraries can generate mocks dynamically, but mocks can also be hand-written.

The important concept is:

**EXAMPLE / FLOW**

```
Mock focuses on interaction.
```

## 32. Stub vs Mock

A useful distinction is:

**EXAMPLE / FLOW**

```
Stub
    →
"Return this when called."

Mock
    →
"Verify this interaction happened."
```

Example:

**EXAMPLE / FLOW**

```
Repository returns existing Gift
    →
Stub behavior
```

**EXAMPLE / FLOW**

```
Receipt sender must be invoked once
    →
Mock/spy interaction
```

## 33. Fake vs Stub

Stub:

**EXAMPLE / FLOW**

```
usually responds with preconfigured data
```

Fake:

**EXAMPLE / FLOW**

```
has a simplified working implementation
```

A fake repository might maintain a real in-memory collection.

A stub repository may simply return one predetermined Gift.



## 34. State-Based Testing

State-based testing verifies final state.

Example:

```C#
gift.Complete();

Assert.Equal(
    GiftStatus.Completed,
    gift.Status);
```

This is often ideal for domain objects because:

```
EXAMPLE / FLOW
behavior
    ↓
observable state
```

is what matters.

## 35. Interaction-Based Testing

Interaction testing verifies calls between collaborators.

Example:

```
EXAMPLE / FLOW
CompleteGiftService
    ↓
must publish exactly one OutboxMessage
```

Test might verify:

```
EXAMPLE / FLOW
OutboxWriter.AddAsync called once
```

Interaction testing is useful when the interaction itself is important behavior.

## 36. Do Not Over-Mock

Bad test:

```
EXAMPLE / FLOW
Mock mapper

Mock validator

Mock repository

Mock clock

Mock logger

Mock formatter
```

```
Mock factory

Mock resolver

Mock event publisher

Mock transaction
```



to test a tiny method.

Possible causes:

**EXAMPLE / FLOW**

```
Class has too many responsibilities

Test is coupled to implementation details

Architecture is overly fragmented
```

A test with 12 mocks is often telling you something about production design.

## 37. Test Behavior, Not Implementation

Suppose implementation currently calls:

**C#**

```
_repository.AddAsync(
    gift);
```

Then you refactor to:

**C#**

```
_repository.SaveAsync(
    gift);
```

If external behavior remains correct, tests should not all break merely because an internal helper changed.

Prefer assertions such as:

**EXAMPLE / FLOW**

```
Gift was persisted

Gift status became Completed

Expected response returned
```

rather than verifying every internal private step.

Microsoft recommends testing behavior and minimizing setup so tests remain resilient to implementation changes.

# PART 3 — TESTABLE C# DESIGN

## 38. Explicit Dependencies

Hard-to-test:

**C#**

```
public sealed class GiftService
{
```

```
    private readonly GiftRepository _repository = new();

    private readonly SystemClock _clock = new();
}
```



Test cannot easily replace those dependencies.

Better:

```C#
public sealed class GiftService
{
    private readonly IGiftRepository _repository;
    private readonly IClock _clock;

    public GiftService(
        IGiftRepository repository,
        IClock clock)
    {
        _repository = repository;

        _clock = clock;
    }
}
```

Now tests control the environment.

## 39. Static Time Is Hard to Test

Production:

```C#
gift.CompletedAt = DateTimeOffset.UtcNow;
```

Test depends on real time.

Instead:

```C#
public interface IClock
{
    DateTimeOffset UtcNow
    {
        get;
    }
}
```

Production:

```C#
public sealed class SystemClock :
    IClock
{
    public DateTimeOffset UtcNow =>
        DateTimeOffset.UtcNow;
}
```

Test:

```C#
public sealed class FakeClock :
    IClock
{
    public DateTimeOffset UtcNow
    {
        get;
        init;
    }
}
```

Now:

```C#
FakeClock clock =
    new()
    {
        UtcNow =
            new DateTimeOffset(
                2026,
                10,
                30,
                12,
                0,
                0,
                TimeSpan.Zero)
    };
```

The test becomes deterministic.

## 40. Randomness Should Be Controllable

Hard:

```C#
Guid id = Guid.NewGuid();
```

when exact generated identity is part of behavior.

Possible abstraction:

```C#
public interface IIdGenerator
{
    Guid NewId();
}
```

Do not abstract every `Guid.NewGuid()` automatically.

Only create the seam where deterministic control matters.

## 41. File System and Network Boundaries

Hard-to-test:

```C#
File.WriteAllText(
    "receipt.txt",
    content);
```

inside business logic.

Better boundary:

```C#
public interface IReceiptStore
{
    Task SaveAsync(
        ReceiptDocument document,
        CancellationToken cancellationToken);
}
```

Infrastructure owns actual files/blob storage.

Application tests can use a fake.

## 42. Pure Functions Are Easy to Test

Example:

```C#
public static decimal CalculateProcessingFee(
    decimal amount,
    decimal percentage)
{
    return amount
        *
        percentage;
}
```

Test:

```C#
[Theory]
[InlineData(100, 0.03, 3)]
[InlineData(200, 0.05, 10)]
public void CalculateProcessingFee_ReturnsExpected(
    decimal amount,
    decimal percentage,
    decimal expected)
{
    decimal actual =
        CalculateProcessingFee(
            amount,
            percentage);

    Assert.Equal(
        expected,
        actual);
}
```

Pure behavior is naturally deterministic.

## 43. Encapsulation Helps Testing

A rich domain object gives you focused behavior:

```C#
gift.AddAllocation(...);

gift.Complete();
```

Tests verify business contracts.

An anemic object with public setters:



```C#
gift.Status = GiftStatus.Completed;
```

has little behavior to test.

The real rules are probably scattered elsewhere.

## 44. Avoid Hidden Global State

Bad:

```C#
public static class CurrentTenant
{
    public static long TenantId
    {
        get;
        set;
    }
}
```

Tests can interfere with each other.

Parallel test execution makes this even more dangerous.

Prefer explicit scoped dependencies.

# PART 4 — DOMAIN TESTING

## 45. Domain Tests Should Be the Easiest Tests

Your Domain layer should generally not require:

```EXAMPLE / FLOW
DI container
Database
HTTP
Configuration
Logging infrastructure
```

Example:

```C#
[Fact]
public void AddAllocation_WhenAmountExceedsRemainingAmount_Throws()
{
    Gift gift =
        Gift.Create(
            tenantId:
                1,
            constituentId:
                100,
            totalAmount:
                100m);

    gift.AddAllocation(
        fundId:
```

```
            10,
        amount:
            75m);

    Assert.Throws<
        InvalidOperationException>(
            () =>
                gift.AddAllocation(
                    fundId:
                        20,
                    amount:
                        30m));
}
```



This directly documents the invariant:

```
EXAMPLE / FLOW
Gift allocations cannot exceed Gift total.
```

## 46. Test Boundary Values

If valid amount is:

```
EXAMPLE / FLOW
amount > 0
```

interesting cases are:

```
EXAMPLE / FLOW
-1
0
0.01
```

not twenty random positive numbers.

For a maximum:

```
EXAMPLE / FLOW
1,000,000
```

test:

```
EXAMPLE / FLOW
999,999.99
1,000,000
1,000,000.01
```

Boundary-value tests are often more valuable than many arbitrary examples.

## 47. Positive and Negative Paths

For every important business rule ask:

```
EXAMPLE / FLOW
What should succeed?

What should fail?
```

For Gift completion:

EXAMPLE / FLOW

```
Fully allocated Gift
    →
Complete

Partially allocated Gift
    →
Reject

Already completed Gift
    →
Reject
```

# PART 5 — APPLICATION SERVICE TESTING

## 48. Application Service Example

C#

```
public sealed class CompleteGiftService
{
    private readonly IGiftRepository _repository;
    private readonly IOutboxWriter _outbox;
    private readonly IUnitOfWork _unitOfWork;

    public CompleteGiftService(
        IGiftRepository repository,
        IOutboxWriter outbox,
        IUnitOfWork unitOfWork)
    {
        _repository = repository;

        _outbox = outbox;

        _unitOfWork = unitOfWork;
    }

    public async Task CompleteAsync(
        CompleteGiftCommand command,
        CancellationToken cancellationToken)
    {
        Gift gift =
            await _repository.FindAsync(
                command.TenantId,
                command.GiftId,
                cancellationToken)
            ??
            throw new
                GiftNotFoundException(
                    command.GiftId);

        gift.Complete();

        await _outbox.AddAsync(
            OutboxMessage.ForGiftCompleted(
                gift),
            cancellationToken);

        await _unitOfWork.SaveChangesAsync(
            cancellationToken);
    }
}
```

## 49. Test Successful Application Flow

C#

```
[Fact]
public async Task CompleteAsync_WhenGiftExists_CompletesAndCommits()
{
    Gift gift =
        GiftTestBuilder
            .CreateFullyAllocatedGift();

    FakeGiftRepository repository =
        new(
            gift);

    SpyOutboxWriter outbox = new();

    SpyUnitOfWork unitOfWork = new();

    CompleteGiftService service =
        new(
            repository,
            outbox,
            unitOfWork);

    await service.CompleteAsync(
        new CompleteGiftCommand(
            gift.TenantId,
            gift.GiftId),
        CancellationToken.None);

    Assert.Equal(
        GiftStatus.Completed,
        gift.Status);

    Assert.Single(
        outbox.Messages);

    Assert.Equal(
        1,
        unitOfWork.SaveCount);
}
```

This verifies:

EXAMPLE / FLOW

```
domain operation happened

outbox intent created

unit of work committed
```

## 50. Test Not-Found Flow

C#

```
[Fact]
public async Task CompleteAsync_WhenGiftMissing_ThrowsAndDoesNotCommit()
{
    FakeGiftRepository repository =
        new(
            gift:
                null);
```

```
    SpyOutboxWriter outbox = new();

    SpyUnitOfWork unitOfWork = new();

    CompleteGiftService service =
        new(
            repository,
            outbox,
            unitOfWork);

    await Assert.ThrowsAsync<
        GiftNotFoundException>(
            () =>
                service.CompleteAsync(
                    new CompleteGiftCommand(
                        1,
                        100),
                    CancellationToken.None));

    Assert.Empty(
        outbox.Messages);

    Assert.Equal(
        0,
        unitOfWork.SaveCount);
}
```

## 51. What Not to Test in Application Unit Tests

If repository is replaced by a fake, this test does **not** prove:

EXAMPLE / FLOW

```
EF Include is correct

SQL query is correct

Tenant filter works in generated SQL

Database transaction works
```

Those belong in integration tests.

Unit tests should not claim more confidence than they actually provide.

# PART 6 — ASP.NET CORE TESTING

## 52. Controller Unit Tests

Suppose:

C#

```
[HttpGet("{giftId:long}")]
public async Task<ActionResult<GiftResponse>>
    Get(
        long giftId,
        CancellationToken cancellationToken)
{
    GiftResponse? response =
        await _query.GetAsync(
            giftId,
```

```
            cancellationToken);

    if (response is null)
    {
        return NotFound();
    }

    return Ok(
        response);
}
```



A controller unit test can verify:

EXAMPLE / FLOW

```
null query result
    →
NotFound
```

but not:

EXAMPLE / FLOW

```
routing
model binding
authentication middleware
real JSON serialization
```

Microsoft explicitly recommends keeping controller unit tests focused on action logic; routing, filters, model binding, and other pipeline behaviors belong in integration tests.

## 53. Controller Unit Test

C#

```
[Fact]
public async Task Get_WhenGiftMissing_ReturnsNotFound()
{
    StubGiftQuery query =
        new(
            result:
                null);

    GiftsController controller =
        new(
            query);

    ActionResult<GiftResponse> result =
        await controller.Get(
            100,
            CancellationToken.None);

    Assert.IsType<
        NotFoundResult>(
            result.Result);
}
```

## 54. ASP.NET Core Integration Testing

For real pipeline behavior use:

Microsoft's current ASP.NET Core integration-testing guidance uses `WebApplicationFactory<TEntryPoint>` to bootstrap the system under test and create an `HttpClient` against a test server.

Example:

```C#
public sealed class GiftsApiTests :
    IClassFixture<
        WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public GiftsApiTests(
        WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task GetGift_WhenMissing_Returns404()
    {
        HttpResponseMessage response =
            await _client.GetAsync(
                "/api/gifts/999999");

        Assert.Equal(
            HttpStatusCode.NotFound,
            response.StatusCode);
    }
}
```

This can exercise:

```EXAMPLE / FLOW
Routing

Middleware

Model binding

Controllers/endpoints

DI

Serialization
```

depending on the test-host configuration.

## 55. What WebApplicationFactory Does Not Mean

Using `WebApplicationFactory` does not automatically mean:

```EXAMPLE / FLOW
production environment is fully reproduced
```

You may still replace:

EXAMPLE / FLOW

```
Database
External HTTP APIs
Authentication
Message broker
```

with test-specific infrastructure.

The test scope must be explicit.

# PART 7 — EF CORE TESTING

## 56. EF Core Testing Requires Special Care

Database behavior includes:

EXAMPLE / FLOW

```
SQL translation

Constraints

Transactions

Collation

Case sensitivity

Provider-specific functions

Concurrency

Indexes

Foreign keys
```

These behaviors cannot always be reproduced by an in-memory collection.

That is why EF Core tests need a deliberate strategy.

## 57. Best Confidence: Test the Real Database Provider

Microsoft's current EF Core guidance recommends having good coverage against the actual production database system and notes that local/containerized database testing can often be practical and fast.

If production uses:

EXAMPLE / FLOW

```
SQL Server
```

then SQL Server integration tests give the strongest confidence for:

EXAMPLE / FLOW

```
query translation
collation behavior
transactions
provider-specific features
raw SQL
```

## 58. Why SQLite Is Not the Same as SQL Server

SQLite can be useful as a lightweight relational fake, but it differs from SQL Server.

Examples Microsoft calls out include:

```
EXAMPLE / FLOW
Different case sensitivity

Different SQL dialect

Unsupported SQL Server-specific functions

Different query behavior
```

Therefore:

```
EXAMPLE / FLOW
Test passes on SQLite
```

does not prove:

```
EXAMPLE / FLOW
Query works identically on SQL Server
```

## 59. EF Core InMemory Provider

A common mistake is treating:

```
EXAMPLE / FLOW
Microsoft.EntityFrameworkCore.InMemory
```

as though it were a relational database.

It is not.

Microsoft currently strongly discourages using the EF Core InMemory provider as a general database fake for application testing because it:

```
EXAMPLE / FLOW
is not relational

does not support transactions normally

does not support raw SQL

has different query behavior
```

and recommends testing against the real database or using other strategies.

## 60. Do Not Mock DbSet Queries

Trying to mock:

```
C#
DbSet<Gift>
```

for query behavior is usually misleading.

Your LINQ expression is supposed to be translated by EF Core into database SQL.

An in-memory mocked collection evaluates:



```
EXAMPLE / FLOW
LINQ-to-Objects
```

instead.

Microsoft specifically advises against mocking `DbSet` query behavior because it cannot reliably reproduce provider translation and database semantics.

## 61. Repository as Test Seam

If Application code depends on:

```
C#
IGiftRepository
```

then a unit test can substitute:

```
C#
FakeGiftRepository
```

and avoid EF entirely.

Microsoft's EF Core testing guidance identifies Repository as the reliable approach for fully stubbing/mocking the data-access layer, with the important trade-off that the repository itself adds implementation and maintenance cost.

## 62. Repository Tests Still Need Real Database Tests

Suppose Application test uses:

```
C#
FakeGiftRepository
```

That proves:

```
EXAMPLE / FLOW
CompleteGiftService works
when repository returns Gift.
```

But the real repository contains:

```
C#
_dbContext.Gifts
    .Include(...)
    .Where(...)
```

That query still needs integration testing.

Therefore:

```
EXAMPLE / FLOW
Repository abstraction
```

does not eliminate:

## 63. Database Test Isolation

Database integration tests must avoid interfering with one another.

Possible strategies include:

```
EXAMPLE / FLOW
Separate database per test suite

Unique schema/database per run

Transaction rollback

Recreate database

Reset known tables

Unique tenant/test IDs
```

The correct technique depends on performance and database behavior.

## 64. Do Not Depend on Test Order

Bad:

```
EXAMPLE / FLOW
Test 1 creates Gift.

Test 2 assumes Gift from Test 1 exists.

Test 3 deletes Gift.
```

Now running Test 2 alone fails.

Tests should normally establish their own required state.

# PART 8 — TEST FIXTURES AND LIFECYCLE

## 65. xUnit Creates Fresh Test-Class Instances

xUnit typically constructs a new test-class instance for each test.

This supports isolation between tests.

For shared expensive resources, use fixtures rather than static mutable state.

xUnit's fixture documentation provides class, collection, and—on current v3—assembly-level fixture mechanisms.

## 66. Class Fixture

Use:

```
C#
IClassFixture<T>
```

when several tests in one class should share an expensive fixture.

Example:

```C#
public sealed class SqlServerFixture :
    IAsyncLifetime
{
    public Task InitializeAsync()
    {
        // Start/setup test database.

        return Task.CompletedTask;
    }

    public ValueTask DisposeAsync()
    {
        // Clean up.

        return ValueTask.CompletedTask;
    }
}
```

Test:

```C#
public sealed class GiftRepositoryTests :
    IClassFixture<
        SqlServerFixture>
{
    private readonly
        SqlServerFixture
        _fixture;

    public GiftRepositoryTests(
        SqlServerFixture fixture)
    {
        _fixture = fixture;
    }
}
```

Current xUnit documentation recommends `IAsyncLifetime`/async disposal support when fixture initialization or cleanup requires asynchronous work.

## 67. Do Not Share Mutable Test State Carelessly

Sharing a fixture may improve performance, but this:

```EXAMPLE / FLOW
Test A changes shared database row.

Test B reads same row simultaneously.
```

creates flaky tests.

Shared infrastructure must have an explicit isolation strategy.

# PART 9 — PARALLEL TESTING

## 68. Tests May Run in Parallel

Modern xUnit supports parallel test execution.

By default, test collections may run concurrently, which means separate test classes can potentially interact with shared resources at the same time.

Therefore this is dangerous:

```C#
public static long CurrentTenantId;
```

shared across tests.

## 69. Parallel-Safe Tests

Good tests should avoid reliance on:

EXAMPLE / FLOW

```
Global static mutable state

Fixed shared file names

Same database row

Same network port

Shared singleton test data
```

If a resource must be shared, use proper fixtures/collections or disable parallelization only for that necessary boundary.

Do not disable all parallel testing merely to hide poor test isolation.

# PART 10 — CODE COVERAGE

## 70. What Is Code Coverage?

Code coverage measures what code was executed while tests ran.

Common metrics include:

EXAMPLE / FLOW

```
Line coverage

Branch coverage

Method coverage
```

Microsoft defines code coverage as measuring how much code is executed during tests and documents built-in/Coverlet-based tooling for .NET.

## 71. Running Coverage

A common .NET command is:

```
C#
dotnet test --collect:"XPlat Code Coverage"
```

Microsoft documents this flow using Coverlet's collector.

## 72. 100% Coverage Does Not Mean Correct Software

Consider:

```
C#
public decimal CalculateFee(
    decimal amount)
{
    return amount * 0.05m;
}
```

Test:

```
C#
[Fact]
public void CalculateFee_Runs()
{
    _ =
        CalculateFee(
            100);
}
```

Coverage:

```
EXAMPLE / FLOW
100% line executed
```

But no correctness was asserted.

Coverage tells you:

```
EXAMPLE / FLOW
what was executed
```

not:

```
EXAMPLE / FLOW
whether behavior was tested meaningfully.
```

Microsoft's testing guidance similarly warns that high coverage does not automatically imply high code quality.

## 73. Branch Coverage

Consider:

```
C#
if (amount > 10_000m)
{
    return MajorGiftFee;
}

return StandardFee;
```

One test with:

```
EXAMPLE / FLOW
amount = 20,000
```

executes:

```
EXAMPLE / FLOW
true branch
```

but never verifies:

```
EXAMPLE / FLOW
false branch.
```

Branch coverage can reveal that.

## 74. What Should You Aim to Cover?

Focus most heavily on:

```
EXAMPLE / FLOW
Business invariants

Complex calculations

Critical application workflows

Security decisions

Data transformations

Error handling

Boundary conditions
```

Simple auto-properties generally need less attention than a complex Gift completion rule.

# PART 11 — TDD

## 75. Test-Driven Development

TDD commonly uses:

```
EXAMPLE / FLOW
Red
  ↓
Green
  ↓
Refactor
```

### Red

Write a failing test describing desired behavior.

```
C#
[Fact]
public void Complete_WhenAllocationsIncomplete_Throws()
{
```

```
    // Test first.
}
```



### Green

Write the simplest production code necessary to make it pass.

### Refactor

Improve the implementation while keeping tests green.

## 76. TDD Is Not the Same as Unit Testing

You can write unit tests:

```
EXAMPLE / FLOW
after code
```

without practicing TDD.

TDD specifically describes the development workflow where tests help drive implementation/design.

Microsoft's .NET testing documentation describes TDD as writing the unit test before the code it verifies.

# PART 12 — BRITTLE AND FLAKY TESTS

## 77. What Is a Brittle Test?

A brittle test fails because implementation details changed even though behavior is still correct.

Example:

```
EXAMPLE / FLOW
Test expects repository method
to be called exactly twice.
```

Implementation becomes more efficient and calls once.

Business result remains correct.

The test fails anyway.

That test is overly coupled to implementation.

## 78. What Is a Flaky Test?

A flaky test sometimes passes and sometimes fails without relevant code changes.

Common causes:

```
EXAMPLE / FLOW
Real current time

Thread timing

Parallel shared state

Randomness

Network dependencies

Database contamination
```

```
Task.Delay assumptions

Unordered collections

Race conditions
```

Flaky tests are particularly damaging because developers stop trusting failures.

## 79. Never Fix Flakiness with Random Sleeps

Bad:

```C#
await Task.Delay(
    5000);
```

to “give the background job enough time.”

Better:

```EXAMPLE / FLOW
Await a deterministic completion signal

Expose controlled dependency

Use TaskCompletionSource in test infrastructure

Poll with bounded timeout when testing true async integration
```

Do not make time guesses your synchronization mechanism.

## 80. Test Timeouts

A timeout can prevent a hanging test suite, but it is not a substitute for deterministic design.

Current xUnit `FactAttribute` includes timeout-related support, but timeouts should be used carefully, especially when tests run under parallel load.

# PART 13 — SECURITY TESTING

## 81. Test Authorization Boundaries

For your multi-tenant Fundraising Platform, important tests include:

```EXAMPLE / FLOW
Tenant A cannot retrieve Tenant B Gift.

Unauthorized user cannot refund Gift.

Reader role cannot modify Campaign.

Tenant identifier cannot be bypassed through query parameter.
```

These are usually:

```EXAMPLE / FLOW
application authorization tests

and/or
```

```
API integration tests
```

depending on where enforcement occurs.

## 82. Repository Tenant Test

Given:

```C#
Task<Gift?> FindAsync(
    long tenantId,
    long giftId,
    CancellationToken cancellationToken);
```

integration test should verify:

```EXAMPLE / FLOW
Gift exists for Tenant 1

Query with Tenant 2

Result = null
```

This tests actual persistence isolation.

## 83. Do Not Put Secrets in Test Data

Avoid committing:

```EXAMPLE / FLOW
Production passwords

API keys

Real access tokens

Real donor information

Production connection strings
```

to test source code.

Use dedicated test credentials and secure CI secrets for integration environments.

# PART 14 — PERFORMANCE AND TEST DESIGN

## 84. Unit Tests Should Be Fast Enough to Run Constantly

Developers should be able to run unit tests during:

```EXAMPLE / FLOW
editing

refactoring

committing

CI
```

If the “unit test” suite requires 20 minutes and a database cluster, it probably contains many integration tests and should be categorized appropriately.



## 85. Integration Tests Can Still Be Fast

Do not assume:

EXAMPLE / FLOW

```
real database
=
too slow
```

Microsoft's EF Core team explicitly notes that its own suite contains tens of thousands of SQL Server tests and recommends considering real database testing rather than automatically reaching for an inaccurate fake.

Correct isolation and efficient setup matter more than the label.

# PART 15 — TEST BUILDERS AND TEST DATA

## 86. The Object Mother Problem

Tests may repeatedly contain:

C#

```
Gift gift =
    Gift.Create(
        tenantId:
            1,
        constituentId:
            100,
        totalAmount:
            100m);

gift.AddAllocation(
    fundId:
        10,
    amount:
        100m);
```

Repeated setup creates noise.

A test builder can help.

## 87. GiftTestBuilder

C#

```
public sealed class GiftTestBuilder
{
    private long _tenantId = 1;

    private long _constituentId = 100;

    private decimal _amount = 100m;

    public GiftTestBuilder
        WithAmount(
            decimal amount)
    {
        _amount = amount;
```

```
        return this;
    }

    public Gift Build()
    {
        return Gift.Create(
            _tenantId,
            _constituentId,
            _amount);
    }
}
```

Test:

```C#
Gift gift =
    new GiftTestBuilder()
        .WithAmount(
            500m)
        .Build();
```

## 88. Builder Must Not Hide Important Setup

Bad:

```C#
Gift gift =
    GiftTestBuilder
        .MagicValidGiftForScenario17();
```

Nobody knows what was configured.

Good test helpers reduce irrelevant setup while keeping scenario-defining values visible.

# PART 16 — WHAT NOT TO TEST

## 89. Do Not Test Framework Code

You normally don't need a unit test proving:

```C#
List<int>.Add()
```

works.

Likewise, don't unit test that:

EXAMPLE / FLOW

```
EF Core's Add() adds an entity

ASP.NET Core routing works generally

HttpClient sends HTTP generally
```

Test **your configuration/use** of those frameworks where needed.

## 90. Do Not Test Private Methods Directly

Suppose:

```C#
private decimal CalculateInternalFee()
```

exists only to support:

```C#
public Receipt CreateReceipt()
```

Usually test:

```EXAMPLE / FLOW
CreateReceipt behavior
```

rather than private implementation.

If a private method contains enough important logic that you desperately want to test it independently, consider whether it represents a separate responsibility that deserves extraction.

## 91. Do Not Test Trivial Auto-Properties

Usually unnecessary:

```C#
[Fact]
public void GiftId_CanBeRead()
{
}
```

unless property behavior itself contains logic.

Focus on meaningful behavior.

# PART 17 — FULL FUNDRAISING TESTING STRATEGY

## 92. Domain Layer

Test heavily with unit tests.

Examples:

```EXAMPLE / FLOW
Gift creation

Gift allocation rules

Gift completion

Campaign state transitions

Fund validity rules

Value object equality
```

No infrastructure.

## 93. Application Layer

Unit-test orchestration using:

```
EXAMPLE / FLOW
Fake repositories

Stub authorization

Fake clock

Spy outbox writer

Fake unit of work
```

Examples:

```
EXAMPLE / FLOW
CompleteGift loads Gift.

Missing Gift throws.

Successful completion creates OutboxMessage.

Failed domain rule does not commit.
```

## 94. Infrastructure Layer

Use integration tests.

Examples:

```
EXAMPLE / FLOW
GiftRepository finds correct Tenant Gift.

Gift allocations load correctly.

SQL Server rowversion detects stale write.

Unique external-reference constraint works.

OutboxMessage transaction works.
```

Prefer the real provider where database behavior matters.

## 95. API Layer

Use focused unit tests only for meaningful controller-specific branching.

Use `WebApplicationFactory` integration tests for:

```
EXAMPLE / FLOW
Routing

Authentication

Authorization

Model binding

Validation
```

```
ProblemDetails

Status codes

Serialization
```

ASP.NET Core's official integration testing guidance uses `WebApplicationFactory` and `TestServer` for this purpose.

## Diagram 1 — Testing Boundaries

EXAMPLE / FLOW

```
DOMAIN
  ↓
Fast Unit Tests
  ↓
Gift rules and invariants


APPLICATION
  ↓
Unit Tests + Test Doubles
  ↓
Use-case orchestration


INFRASTRUCTURE
  ↓
Integration Tests
  ↓
EF Core + SQL Server + external adapters


API
  ↓
WebApplicationFactory
  ↓
HTTP pipeline integration


SYSTEM
  ↓
Small set of E2E tests
```

# PART 18 — COMMON MISTAKES

### Mistake 1 — Unit Test Uses Real Database

C#

```
[Fact]
public async Task Complete_Works()
{
    await sqlServer...
}
```

This may be a useful test, but call it:

EXAMPLE / FLOW

```
integration test
```

not unit test.

Correct categorization helps maintain test expectations.



## Mistake 2 — Mock Everything

Mocks are tools, not goals.

Prefer simple real objects when they are:

EXAMPLE / FLOW

```
fast
deterministic
dependency-free
```

You do not need to mock `Gift`.

Use the real domain object.

## Mistake 3 — Testing Implementation Calls

Bad:

EXAMPLE / FLOW

```
Verify private helper A called

Verify helper B called

Verify mapper called twice

Verify list enumerated once
```

unless those interactions are contractual behavior.

## Mistake 4 — One Giant Test

Bad:

EXAMPLE / FLOW

```
Create Gift
Complete Gift
Generate Receipt
Refund Gift
Delete Gift
```

in one test.

Failure diagnosis becomes difficult.

## Mistake 5 — Shared Mutable Static Data

Especially dangerous because xUnit can run test collections concurrently.

## Mistake 6 — async void Tests

Use:

C#

```
async Task
```

so asynchronous failures are observed.

## Mistake 7 — `.Result` and `.Wait()`

Use:

for async production code.

### Mistake 8 — Testing EF Queries with InMemory Provider as Proof of SQL Correctness

The EF InMemory provider is not a relational database and is specifically discouraged for general query-testing purposes.

### Mistake 9 — SQLite Test Assumed Equivalent to SQL Server

Different database providers have different SQL, collation, translation, and feature semantics.

### Mistake 10 — Chasing 100% Coverage

Coverage should reveal gaps, not become a vanity metric.

### Mistake 11 — Random Test Data Everywhere

If every test uses random inputs:

EXAMPLE / FLOW

```
Failure
    ↓
Which value caused it?
```

Use fixed deterministic inputs unless property-based/randomized testing is deliberately being used.

### Mistake 12 — Time-Based Sleeps

Avoid:

C#

```
Thread.Sleep(...)
```

and arbitrary `Task.Delay(...)` for coordination.

## PART 19 — BEST PRACTICES

A strong test suite should:

EXAMPLE / FLOW

```
Test observable behavior.

Keep unit tests fast.

Keep unit tests isolated.

Use deterministic time and data.

Use Fact for fixed scenarios.

Use Theory for data-driven scenarios.

Use meaningful names.

Keep Arrange minimal.

Avoid logic inside tests.

Use real domain objects.
```

```
Replace infrastructure at unit-test boundaries.

Use fakes/stubs instead of mocks where simpler.

Mock interactions only when interactions matter.

Await async code.

Test exceptions explicitly.

Test important boundary values.

Use real SQL Server integration tests for important EF behavior.

Use WebApplicationFactory for HTTP pipeline tests.

Keep tests independent from execution order.

Design tests to survive parallel execution.

Use code coverage as a diagnostic metric, not a quality score.

Keep test helpers readable.

Treat flaky tests as defects.

Test security boundaries explicitly.

Run tests continuously in CI.
```

# PART 20 — PRACTICAL EXERCISES

## Exercise 1 — Basic Domain Test

Write tests for:

```
C#
Gift.Create(...)
```

covering:

```
EXAMPLE / FLOW
Positive amount → succeeds

Zero amount → throws

Negative amount → throws
```

Use `[Theory]` where appropriate.

## Exercise 2 — Allocation Boundary

Given:

```
EXAMPLE / FLOW
Gift total = $100
```

test:

EXAMPLE / FLOW

```
Allocate $99.99 → allowed

Allocate $100 → allowed

Allocate $100.01 → rejected
```

## Exercise 3 — Complete Gift

Test:

EXAMPLE / FLOW

```
Fully allocated Pending Gift
    →
Completed

Partially allocated Gift
    →
Exception

Already Completed Gift
    →
Exception
```

## Exercise 4 — Stub Repository

Create:

C#

```
StubGiftRepository
```

that returns:

EXAMPLE / FLOW

```
one configured Gift
```

and use it to test:

C#

```
CompleteGiftService
```

## Exercise 5 — Spy Outbox

Create:

C#

```
SpyOutboxWriter
```

and verify:

EXAMPLE / FLOW

```
successful Gift completion
    →
exactly one GiftCompleted OutboxMessage
```

## Exercise 6 — Fake Clock

Replace:

```C#
DateTimeOffset.UtcNow
```

with:

```C#
IClock
```

and assert an exact known completion timestamp.

## Exercise 7 — Async Exception

Write:

```C#
Assert.ThrowsAsync<GiftNotFoundException>
```

for a missing Gift.

## Exercise 8 — SQL Server Repository Integration Test

Seed:

```
EXAMPLE / FLOW
Tenant 1 Gift 100

Tenant 2 Gift 200
```

verify:

```
EXAMPLE / FLOW
Tenant 1 cannot load Tenant 2 Gift.
```

Use the real SQL Server provider.

## Exercise 9 — API Test

Using:

```C#
WebApplicationFactory<Program>
```

verify:

```
EXAMPLE / FLOW
GET /api/gifts/999999
   →
404
```

through the actual HTTP pipeline.

## Exercise 10 — Theory Conversion

Refactor three almost-identical tests into:

```C#
[Theory]
[InlineData(...)]
```

without using loops inside the test.

### Exercise 11 — Coverage Review

Generate coverage:

```C#
dotnet test --collect:"XPlat Code Coverage"
```

Then identify:

```EXAMPLE / FLOW
important untested branch
```

rather than simply trying to increase the percentage.

### Exercise 12 — Flaky Test Review

Find any test using:

```EXAMPLE / FLOW
DateTime.Now

Thread.Sleep

random data

shared static collection

fixed database row IDs
```

and make it deterministic.

## PART 21 — EXACTLY 25 INTERVIEW QUESTIONS

### 1. What is a unit test?

A unit test verifies a small unit of observable behavior in isolation from external infrastructure.

A good unit test should generally be:

```EXAMPLE / FLOW
fast
isolated
repeatable
self-checking
```

For example, testing `Gift.Complete()` should not require SQL Server or ASP.NET Core.

Microsoft's current guidance emphasizes exactly these characteristics.

### 2. What is the difference between unit testing and integration testing?

A unit test isolates one component and normally substitutes external dependencies.

An integration test verifies that several real components work together.

Example:

```EXAMPLE / FLOW
Gift.Complete()
    →
Unit test
```

while:

EXAMPLE / FLOW

```
GiftRepository
    +
EF Core
    +
SQL Server
```

is an integration test.

ASP.NET Core pipeline tests using `WebApplicationFactory` are also integration tests.

## 3. What is the AAA pattern?

AAA means:

EXAMPLE / FLOW

```
Arrange
Act
Assert
```

Arrange prepares the system under test and input.

Act executes the behavior.

Assert verifies the observable result.

Example:

C#

```
Gift gift =
    GiftTestBuilder
        .CreateValidGift();

gift.Complete();

Assert.Equal(
    GiftStatus.Completed,
    gift.Status);
```

## 4. What makes a good unit-test name?

A good name tells you:

EXAMPLE / FLOW

```
behavior/method

scenario

expected result
```

Example:

EXAMPLE / FLOW

```
Complete_WhenAllocationsAreIncomplete_Throws
```

Microsoft recommends names that communicate the method, scenario, and expected behavior.

## 5. What is the difference between [Fact] and [Theory] in xUnit?

`[Fact]` represents a test with one fixed scenario.

`[Theory]` is data-driven and runs the same test logic for one or more supplied data rows.

Example:



```C#
[Theory]
[InlineData(0)]
[InlineData(-1)]
```

is useful when several inputs should follow the same rule.

## 6. What is a test double?

A test double is a replacement for a production dependency used during testing.

Common categories include:

```EXAMPLE / FLOW
Dummy
Stub
Fake
Spy
Mock
```

Each has a different purpose.

For example:

```EXAMPLE / FLOW
Stub
    →
return controlled value

Spy
    →
record calls

Fake
    →
lightweight working implementation
```

## 7. What is the difference between a stub and a mock?

A stub normally provides controlled answers:

```EXAMPLE / FLOW
"When FindAsync is called,
return this Gift."
```

A mock focuses on verifying interactions:

```EXAMPLE / FLOW
"Verify SendReceiptAsync
was called once."
```

Stub-based tests are usually state/result-oriented.

Mocks are commonly used for interaction-oriented behavior.

## 8. What is the difference between a fake and a mock?

A Fake has a simplified working implementation.

For example:



EXAMPLE / FLOW

```
FakeGiftRepository
```

might maintain an in-memory list.

A Mock is primarily configured to verify expected interactions.

Fakes are often easier to understand when a dependency has simple behavior.

## 9. Why is dependency injection useful for testability?

Dependency injection makes dependencies explicit and replaceable.

Production:

C#

```
GiftService(
    IGiftRepository repository,
    IClock clock)
```

Test can provide:

EXAMPLE / FLOW

```
FakeGiftRepository
FakeClock
```

instead of requiring production infrastructure.

## 10. Why should DateTime.UtcNow often be abstracted in important business logic?

Real current time makes tests non-deterministic.

Injecting:

C#

```
IClock
```

lets a test control the exact timestamp.

This makes behavior repeatable and removes timing-based failures.

Do not abstract time everywhere automatically; do so where time is part of meaningful behavior.

## 11. How do you test asynchronous methods?

Make the test asynchronous:

C#

```
[Fact]
public async Task CreateAsync_WhenValid_Succeeds()
{
    await service.CreateAsync(...);
}
```

Avoid:

EXAMPLE / FLOW

```
async void
```

```
.Result
*
.Wait()
```



Tests should await the operation so completion and exceptions are observed correctly.

## 12. How do you test an asynchronous exception with xUnit?

Use:

```C#
await Assert.ThrowsAsync<
    GiftNotFoundException>(
        () =>
            service.GetAsync(...));
```

Current xUnit v3 `ThrowsAsync<T>` verifies the expected exception from an asynchronous delegate and returns it for further assertions.

## 13. What is a brittle test?

A brittle test depends too strongly on implementation details.

For example, a test may verify:

```
EXAMPLE / FLOW
helper method called exactly twice
```

even though that call count has no business significance.

A harmless refactor then breaks the test.

Good tests prefer observable behavior over internal implementation sequencing.

## 14. What is a flaky test?

A flaky test produces inconsistent results without relevant production-code changes.

Common causes include:

```
EXAMPLE / FLOW
shared state
timing
real clock
network
database contamination
race conditions
random inputs
parallel test execution
```

Flaky tests should be treated as defects because they damage confidence in the suite.

## 15. Should every class be mocked in a unit test?

No.

Use real objects when they are:

```
EXAMPLE / FLOW
fast
deterministic
dependency-free
```

For example, use a real `Gift` when testing an application service.

Replace only dependencies that introduce boundaries or uncontrollable b



Over-mocking often couples tests to implementation and can reveal overly fragmented production design.

## 16. Should private methods be tested directly?

Usually no.

Test the public behavior that depends on the private implementation.

If a private method contains important standalone behavior that is difficult to test through the public API, consider whether that behavior deserves extraction into its own component.

## 17. Why is the EF Core InMemory provider not a reliable replacement for SQL Server tests?

It is not a relational database and differs significantly from SQL Server.

Microsoft notes limitations including:

**EXAMPLE / FLOW**

```
no normal transaction support
no raw SQL
different query semantics
fewer relational behaviors
```

and currently discourages using it as a general database fake for application testing.

## 18. Is SQLite in-memory equivalent to SQL Server for EF Core testing?

No.

SQLite is relational and can be useful, but provider behavior differs.

Examples include:

**EXAMPLE / FLOW**

```
case sensitivity

SQL syntax

provider-specific functions

translation capabilities
```

Therefore passing SQLite tests do not guarantee SQL Server correctness.

## 19. Should DbSet queries be mocked?

Usually not.

EF LINQ queries are translated by the provider.

A mocked `DbSet` backed by an in-memory collection executes LINQ-to-Objects instead of testing actual SQL translation.

Microsoft specifically discourages this approach for query testing.

## 20. How should Repository code be tested?

Use two different levels.

Application code that depends on:

```C#
IGiftRepository
```

can use a fake/stub repository in unit tests.

The concrete:

```C#
GiftRepository
```

should have integration tests against the actual database/provider because its important behavior includes EF query translation and persistence behavior.

## 21. How should ASP.NET Core controllers be tested?

Unit-test controller-specific decisions only.

For example:

```EXAMPLE / FLOW
null result
    →
NotFound
```

Do not use controller unit tests to claim that routing, model binding, filters, or middleware work.

Microsoft recommends integration tests for those framework-level interactions.

## 22. What is `WebApplicationFactory<TEntryPoint>`?

`WebApplicationFactory<TEntryPoint>` bootstraps an ASP.NET Core application for integration testing and provides an `HttpClient` connected to a test server.

It allows tests to exercise:

```EXAMPLE / FLOW
routing
middleware
DI
controllers/endpoints
serialization
```

without manually hosting a real external web server.

## 23. Does 100% code coverage mean the application is fully tested?

No.

Coverage shows that code executed.

It does not prove:

```EXAMPLE / FLOW
correct assertions

correct business cases

security correctness

boundary correctness

missing requirements
```

A test can execute every line and assert nothing useful.

Use coverage to find suspicious gaps rather than treating the percentage



## 24. What is TDD?

TDD means Test-Driven Development.

The common cycle is:

EXAMPLE / FLOW

```
Red
    →
write failing test

Green
    →
write minimum code to pass

Refactor
    →
improve design while tests remain green
```

TDD is a development workflow.

Unit testing is a testing technique.

You can use unit tests without practicing TDD.

## 25. How would you test the Fundraising Platform?

Use a layered testing strategy.

### Domain

Unit-test:

EXAMPLE / FLOW

```
Gift invariants
Allocation rules
Campaign/Fund behavior
Value objects
```

### Application

Unit-test with:

EXAMPLE / FLOW

```
Fake repositories
Fake clock
Stub authorization
Spy Outbox writer
Fake Unit of Work
```

### Infrastructure

Integration-test against SQL Server:

EXAMPLE / FLOW

```
Repository queries
Tenant filtering
Transactions
Concurrency
```

```
EF mappings
Outbox persistence
```



## API

Use:

```C#
WebApplicationFactory<Program>
```

for:

EXAMPLE / FLOW

```
HTTP routes
authentication
authorization
validation
ProblemDetails
serialization
status codes
```

## Critical system flows

Keep a smaller end-to-end suite for scenarios such as:

EXAMPLE / FLOW

```
Create Gift
    ↓
Complete Gift
    ↓
Persist Outbox
    ↓
Publish integration message
```

The important rule is:

> **Test business behavior at the cheapest reliable level, but test technology-specific behavior using the real technology boundary that matters.**

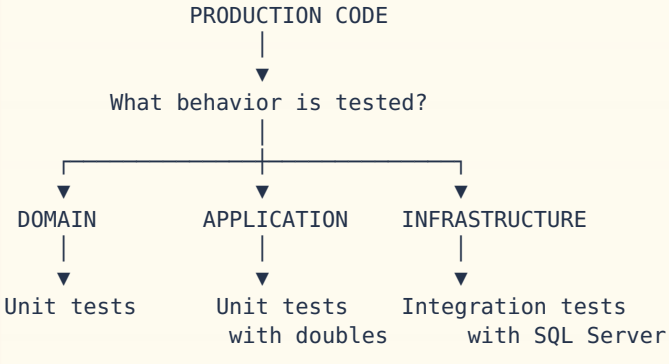
# PART 22 — QUICK REVISION

| Concept | Meaning |
|---|---|
| Unit test | Tests small behavior in isolation |
| Integration test | Tests components working together |
| E2E test | Tests complete system workflow |
| AAA | Arrange, Act, Assert |
| Fact | One fixed test case |
| Theory | Data-driven test |
| InlineData | Simple theory input |
| MemberData | More complex theory data |
| Test double | Replacement dependency |

| Concept | Meaning |
|---|---|
| Dummy | Required but unused object |
| Stub | Returns controlled results |
| Fake | Simplified working implementation |
| Spy | Records interactions |
| Mock | Verifies expected interactions |
| State testing | Verify final state/result |
| Interaction testing | Verify collaboration |
| Testability | Ability to control dependencies |
| Fake clock | Deterministic time |
| `ThrowsAsync` | Async exception assertion |
| Fixture | Shared test context/resource |
| Parallel tests | Multiple tests may execute concurrently |
| Code coverage | Measures executed code |
| TDD | Red → Green → Refactor |
| Flaky test | Non-deterministic test |
| `WebApplicationFactory` | ASP.NET Core integration test host |

# PART 23 — COMPLETE TESTING FLOW FOR THE FUNDRAISING PLATFORM

**EXAMPLE / FLOW**




API:

**EXAMPLE / FLOW**

```
HTTP endpoint
    ↓
WebApplicationFactory
    ↓
Middleware
    ↓
Authentication
    ↓
Authorization
    ↓
```

```
Routing
    ↓
Controller
    ↓
Application
```

Critical production flow:

**EXAMPLE / FLOW**

```
Create Gift
    ↓
Domain invariants
    ↓
Repository
    ↓
SQL Server
    ↓
OutboxMessage
    ↓
Background publisher
```

Test it at several levels:

**EXAMPLE / FLOW**

```
Gift invariant
    →
Unit test

CompleteGiftService orchestration
    →
Unit test

GiftRepository SQL behavior
    →
Integration test

POST /gifts/{id}/complete
    →
API integration test

Complete deployment workflow
    →
Small E2E suite
```

## FINAL MENTAL MODEL

When writing a test, ask:

**EXAMPLE / FLOW**

```
1. What behavior am I proving?

2. What is the smallest useful test boundary?

3. Which dependencies must be real?

4. Which dependencies should be controlled?

5. Is this really a unit test or an integration test?

6. Could this test run independently?

7. Could it run in parallel?
```

```
8. Is time/randomness deterministic?

9. Am I testing behavior or implementation details?

10. Would this test still be valuable after a refactor?
```

Then choose:

**EXAMPLE / FLOW**

```
Pure business behavior
    →
Unit test

Application orchestration
    →
Unit test + test doubles

EF/database behavior
    →
Real database integration test

ASP.NET Core pipeline behavior
    →
WebApplicationFactory integration test

Complete deployed workflow
    →
End-to-end test
```

The central lesson is:

**Good testing is not about writing a test for every method or reaching 100% coverage. It is about creating fast, trustworthy feedback around important behavior. Keep domain tests pure, use dependency injection to control application dependencies, test EF Core against the database behavior that matters, test the ASP.NET Core pipeline through integration tests, and design production code so important decisions are deterministic and observable.**

**Next roadmap topic: Topic 37 — ASP.NET Core Fundamentals and Application Startup.**